



Master 1 SD, SIME
Master 2 SIME

Apprentissage profond



Nom :
Prénom :

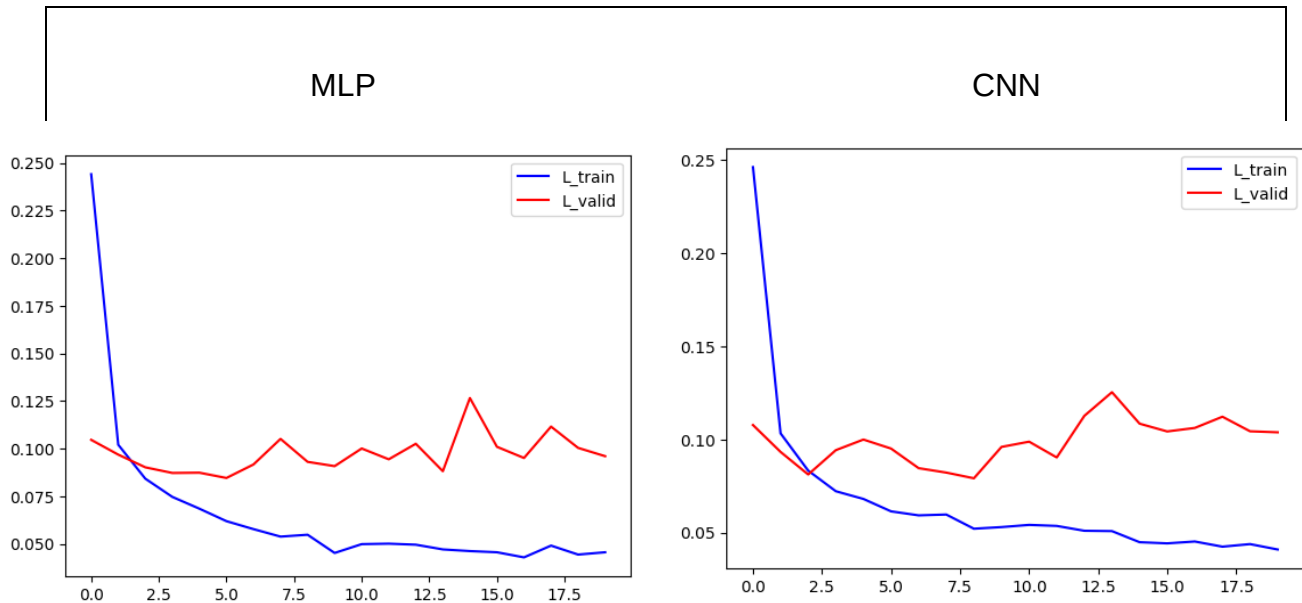
Séance de TP 3 Réseaux Convolutifs – GPU et Tensorboard

- 1- Comparer les temps d'exécution sur CPU et GPU pour l'apprentissage du MLP ayant la meilleure configuration obtenue lors de la séance précédente. Relever le nombre de batch traités par seconde.

The results show that the MLP execution on the GPU was significantly faster than on the CPU. While the GPU version took 1min37s to complete, with an average time of 4.89s/it, the CPU version took 3min57s, with an average time of 11.88s/it. This demonstrates a considerable speedup when using the GPU, which is optimized for matrix operations and parallel calculations, essential for neural networks.

Although MLPs are not as demanding as CNNs in terms of graphics processing, the GPU still offers a significant performance gain due to its ability to handle batch operations more efficiently. The CPU, on the other hand, is better suited for sequential tasks and may be limited by the smaller number of cores specialized for parallel computing.

2- Comparer les performances en test, nombre de paramètres libres et temps d'exécution avec un CNN par rapport au MLP précédent.



The results show that CNN outperformed MLP in terms of test accuracy, reaching 98.39% against 97.44%, we can see that both methods obtained values close to Loss. This improvement in image recognition is expected, since CNNs are specifically designed to extract relevant spatial features thanks to their convolution and pooling layers. On the other hand, MLP, which treats images as flattened vectors, cannot exploit spatial relationships between pixels as effectively, which explains its slight inferiority in terms of performance.

Regarding the number of parameters, CNN has much fewer (20,490) compared to MLP (468,874), which means that it is more efficient in terms of storage and computation, thus reducing the risk of overfitting. However, this efficiency does not necessarily translate into faster execution time: CNN takes slightly longer per iteration (5.43s vs. 4.89s), likely due to the convolution operations, which are more complex than the matrix multiplications used in MLP. Despite this slight increase in execution time, the improved accuracy and reduced parameters make CNN a more optimal choice for image classification.

- 3-** Envisager différentes architectures de réseaux convolutifs pour atteindre les meilleures performances possibles sur la base Fashion. Restituer les architectures utilisées, les courbes d'apprentissage et les performances grâce à TensorBoard.

When testing the initial CNN on the Fashion MNIST dataset, the results were significantly lower than those obtained with Digits MNIST, with an accuracy rate of only 41.65%. This performance suggests that the original architecture was not suitable for recognizing the more complex patterns of clothing, which have less distinctive textures and shapes than handwritten digits. The reduced number of parameters (20,490) may have limited the model's ability to learn richer representations of the data.

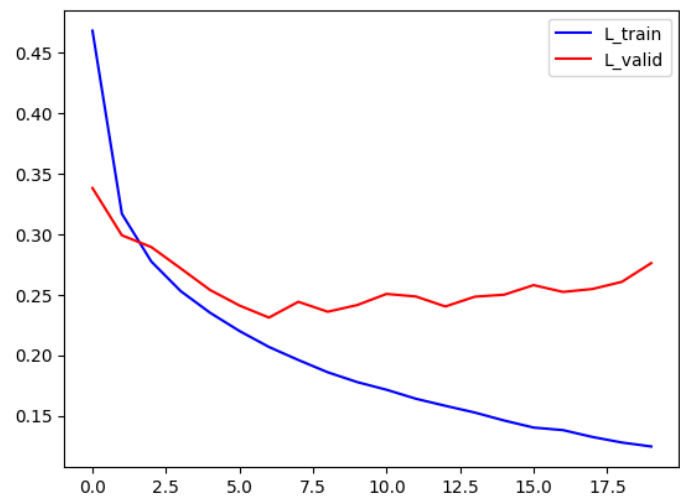
To improve the results, two other deeper architectures were tested, DEEPCNN and CNNBatchNorm. DEEPCNN showed a slight improvement in accuracy, reaching 43.29%, but it was still well below expectations, despite having a much larger number of parameters (467,818). CNNBatchNorm, on the other hand, brought a drastic improvement, reaching 91.11% accuracy, indicating that batch normalization helped the model generalize better to the Fashion MNIST data. This architecture also required fewer epochs to converge, stabilizing in the fifth epoch.

Another approach tested was ResNetLikeCNN, which achieved 90.86% accuracy with a much smaller number of parameters (96,554). This result suggests that the introduction of residual shortcuts improved learning without the need for an excessive increase in model complexity. Furthermore, tests with different hyperparameters such as batch size, number of epochs, and learning rate did not significantly affect the final accuracy, reinforcing the importance of the architecture in obtaining good results.

We can see that the validation loss of the CNNBatchNorm and ResNetLikeCNN architectures showed a large increase as the epochs passed, which is a sign of overfitting, but even so, their initial epochs (before overfitting) obtained the best accuracy in testing.

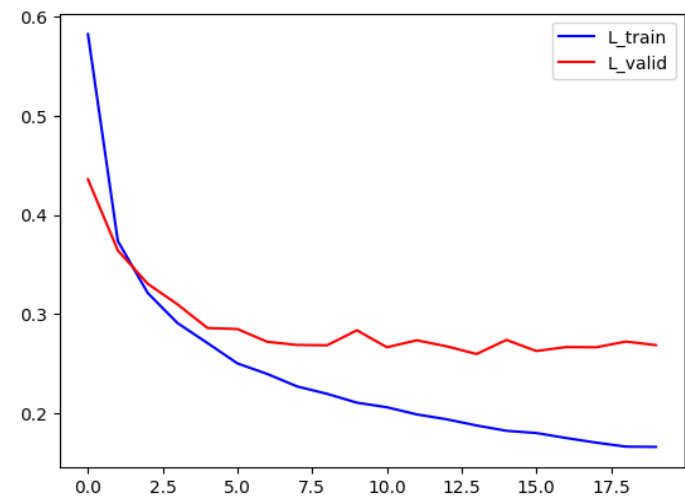
Original CNN

Taux de reco en Test: 41.65 % de réussite
Meilleure époque : 6
Nombre de paramètres libres: 20490



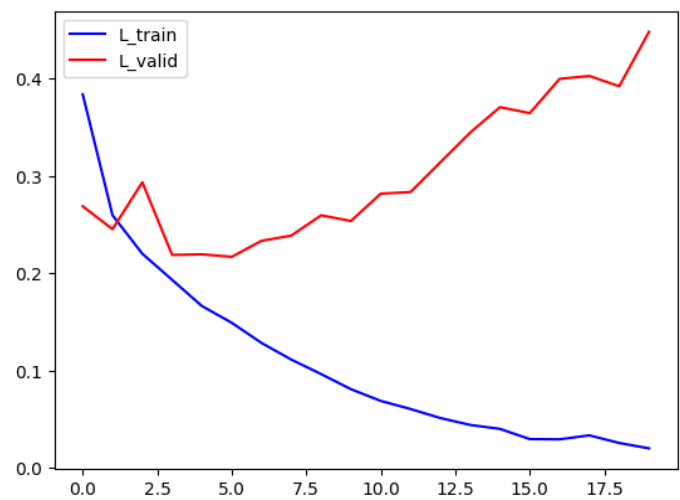
DEEPCNN

Taux de reco en Test: 43.29 % de réussite
Meilleure époque : 13
Nombre de paramètres libres: 467818



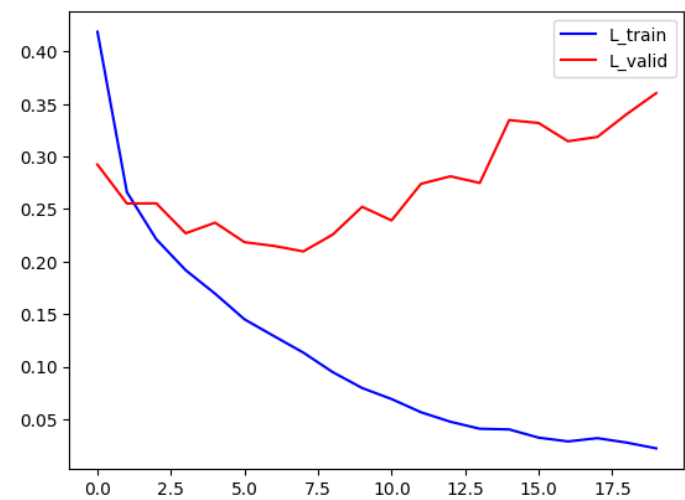
CNNBatchNorm

Taux de reco en Test: 91.11 % de réussite
Meilleure époque : 5
Nombre de paramètres libres: 421834



ResNetLikeCNN

Taux de reco en Test: 90.86 % de réussite
Meilleure époque : 7
Nombre de paramètres libres: 96554



```

import torch.nn.functional as F

class DeepCNN(nn.Module):
    def __init__(self):
        super(DeepCNN, self).__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(0.25)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(0.25)
        )
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.dropout(x, 0.5, training=self.training)
        x = self.fc2(x)
        return x

```

```

class CNNBatchNorm(nn.Module):
    def __init__(self):
        super(CNNBatchNorm, self).__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

```

```

self.fc1 = nn.Linear(64 * 7 * 7, 128)
self.fc2 = nn.Linear(128, 10)

def forward(self, x):
    x = self.conv1(x)
    x = self.conv2(x)
    x = torch.flatten(x, 1)
    x = F.relu(self.fc1(x))
    x = self.fc2(x)
    return x

```

```

class ResNetLikeCNN(nn.Module):
    def __init__(self):
        super(ResNetLikeCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(64, 64, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(64 * 7 * 7, 10)

    def forward(self, x):
        x = F.relu(self.bn1(self.conv1(x)))
        x_res = x
        x = F.relu(self.bn1(self.conv2(x)))
        x += x_res  # Skip connection

        x = self.pool(x)

        x = F.relu(self.bn2(self.conv3(x)))
        x_res = x
        x = F.relu(self.bn2(self.conv4(x)))
        x += x_res  # Skip connection

        x = self.pool(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)
        return x

```